

Axonrouter – AzureML usage and information

Folder structure

1. Folder Structure:

```
AxonRouter/  
├─ data/  
├─ notebooks/  
│   ├── train_and_eval.ipynb  
│   └─ inference_demo.ipynb  
├─ src/  
│   └─ axonrouter/  
│       ├── preprocessing/  
│       ├── models/  
│       ├── inference/  
│       ├── training/  
│       └─ cli.py  
├─ environment.yml  
├─ Dockerfile  
├─ .github/  
│   └─ workflows/  
│       └─ azureml-train-deploy.yml  
├─ train.py  
├─ evaluate.py  
└─ register.py
```

2. Environments:

```
channels:  
  - conda-forge  
  - defaults  
dependencies:  
  - python=3.9  
  - tensorflow=2.12.*  
  - ipykernel  
  - imageio  
  - matplotlib  
  - numpy  
  - pandas  
  - scikit-image  
  - scipy  
  - networkx  
  - skan  
  - pybullet  
  - sqlite  
  - pytz  
  - scikit-learn  
  - pip  
  - pip:  
    - opencv-python-headless  
    - typer  
    - azure-identity  
    - azure-ai-ml  
    - azureml-core  
    - azureml-defaults  
    - patchify  
name: cv_env_azure
```

Environment

Code to create the environments:

Training Process

```
FROM mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04
COPY environment.yml .
RUN conda env create -f environment.yml
```

The model training pipeline is defined in `train.py` and is designed to train a root segmentation model using grayscale root plate images and corresponding binary root masks. It includes preprocessing, patchification, and training using a U-Net-based Keras model.

Script: `train.py` The script is CLI-compatible and receives its inputs as command-line arguments. It supports both local execution and cloud execution via Azure ML.

Inputs

- `--pretrained-model`: Path to a pretrained `.h5` or `.keras` model. Can be a file or a directory.
- `--data-root`: Folder containing the subdirectories `train_images/`, `train_masks/`, `val_images/`, and `val_masks/`.
- `--model-dir`: Output directory where the trained model will be saved.
- `--patch-size`: Size of square patches to train on (default: 256).
- `--batch-size`: Number of patches per batch (default: 16).
- `--epochs`: Number of training epochs (default: 1; typical usage: 15–50).

Data Preprocessing

1. Images are loaded in grayscale and normalized to `[0, 1]`.
2. Masks are loaded as binary maps where root pixels `> 0`.
3. Images are stacked into RGB format by duplicating the grayscale channel.

4. Both images and masks are patchified into non-overlapping tiles of size `patch_size × patch_size`.
5. Any pair that fails to load or has inconsistent dimensions is skipped with a warning.
6. Each full-size image typically yields dozens of patches, greatly increasing the dataset size.

Model Loading

- The script loads a Keras model from a specified file (supports both .h5 and .keras formats).
- It supports custom metrics like a custom F1 score implemented using Keras backend functions.
- If the pretrained model path is a directory, the first compatible file is used.

Training Loop

Model is trained using:

- Binary crossentropy or loss inherited from the pretrained model.
- Custom F1 metric for performance evaluation.

EarlyStopping is used to prevent overfitting based on validation loss.

Training and validation are performed on the extracted image/mask patches.

Model is saved to the specified output directory as `model.keras`.

Example output directory structure:

`model-dir/`

└── `model.keras`

Sample CLI Command (Local)

```
python train.py \
```

```
--pretrained-model pretrained/model.keras \
```

```
--data-root ./data/axonrooter_dataset \
```

--model-dir ./outputs/trained_model \

--patch-size 256 \

--batch-size 16 \

--epochs 30

Training is executed as part of an Azure ML pipeline job. Below are the specifics from a production run:

- Experiment: axonrooter_weekly_retrain
- Job ID: d1c34a1f-7816-4699-b47c-1ba0957cb728
- Compute target: adsai-lambda-2
- Trigger: CI/CD workflow on GitHub push
- Inputs:
 - data_root: azureml:axonrooter_student_corrections_Endijs_20250626_040517:1
 - pretrained_model:
azureml:azureml_heroic_orange_wy4yx9gy4w_input_data_train_pretrained_model:1
- Output:
 - model: azureml:azureml_b5d270d2-ed3a-4bbd-b448-0be71d18c460_output_data_model:1

Evaluation

The model evaluation pipeline is defined in `evaluate.py`. It computes performance metrics on a set of test image-mask pairs using patchified inference and binary classification evaluation.

Script: `evaluate.py` The script is CLI-compatible and is used in both local validation and cloud-based evaluation jobs in Azure ML pipelines.

Inputs

- --model_path: Path to the directory containing model.keras.

- `--test_data_dir`: Folder containing `test_images/` and `test_masks/`.
- `--accuracy_path`: Output directory where `metrics.json` and visualizations are saved.
- `--patch_size`: Size of square patches to use during evaluation (default: 256).

Evaluation Logic

Test images and masks are loaded and patchified into 256×256 tiles. The model is loaded with the custom F1 metric. Binary predictions are generated by thresholding outputs at 0.5. F1 score and accuracy is computed by flattening all prediction and ground truth arrays. Results are saved to a `metrics.json` file.

Visualizations of the first five prediction results are also saved to a subfolder called `pred_visuals`.

Sample CLI Command (Local)

```
python evaluate.py
```

```
--model_path ./outputs/trained_model
```

```
--test_data_dir ./data/test_dataset
```

```
--accuracy_path ./outputs/eval_results
```

```
--patch_size 256
```

Azure ML Job Details

Experiment: axonrooter_weekly_retrain Job ID: db8f9913-a32a-4ad7-af14-9b96887cf3e5 Compute target: adsai-lambda-2 Input model: azureml_b5d270d2-ed3a-4bbd-b448-0be71d18c460_output_data_model:1 Input data: bigger-test-segmentation-dataset-only-roots:1 Output: azureml_89fb3527-ad06-489f-b2ef-4fb04e30ccc3_output_data_accuracy:1

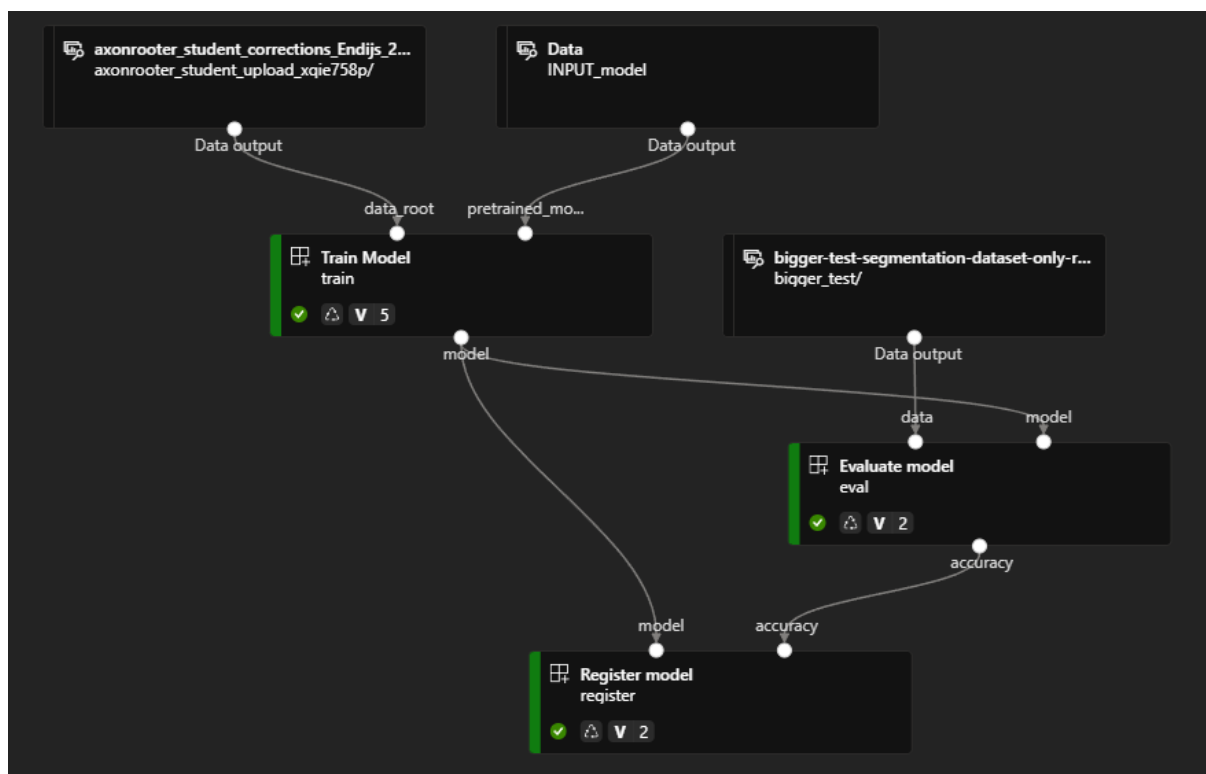
Model Registration

Model registration is handled through `register.py`, which is responsible for validating evaluation results and saving successful models into the Azure ML model registry. This step ensures version control, traceability, and accessibility of models for future inference or deployment stages. The script reads the evaluation metrics from a `metrics.json` file, which is generated during the evaluation phase, and checks whether a usable performance metric like F1 score or accuracy is present.

The script is executed as a pipeline step in the `axonrooter_weekly_retrain` endpoint. It receives the trained model and its corresponding evaluation output as inputs. Although the registration logic includes optional thresholds, the current setup registers all models regardless of score, primarily to support debugging and auditability. Models are saved under the name `axonrooter-finetuned` with tags that include the performance score used during registration.

Deployment

Whole CI/CD/CT pipeline



Monitoring

The model is deployed using a custom scoring script defined in `score.py`. This script enables inference via Azure Managed Online Endpoints, taking grayscale root plate images as input and returning binary root segmentation masks as output. It supports image patching, model prediction, and output reassembly to maintain compatibility with large image sizes.

Initialization is handled in the `init()` function, which locates and loads a `.keras` model file using a custom F1 metric. During inference, the `run()` function decodes a base64-encoded grayscale PNG image, resizes it via zero-padding to ensure divisibility by 256, and splits it into 256×256 patches. Each patch is normalized, converted to RGB format, and passed to the model for prediction.

The resulting patch-level binary predictions are unpatchified to reconstruct the full image mask. Any added padding is removed, and the final binary mask is returned as a base64-encoded PNG. The service supports single-image inference per request and is suitable for real-time or batch segmentation scenarios within Azure ML.

Data Assets

In the AxonRouter project, all datasets are managed as versioned Azure ML Data Assets, which ensures traceability, reproducibility, and a clear separation between data, code, and compute. This approach is central to the MLOps workflow. The primary data assets in the workspace include:

Initial Training & Testing Datasets: These are curated, baseline datasets containing high-quality images and masks used for initial model training and standardized evaluation. They follow a predefined structure with `train_images/`, `train_masks/`, `val_images/`, and `val_masks/` subdirectories. An example is the `bigger-test-segmentation-dataset-only-roots` asset used in the evaluation pipeline.

User-Submitted Correction Datasets: A key feature of the Continuous Training (CT) pipeline is the ability to create new data assets from corrections submitted by users through the Streamlit web application. When a user uploads new images and corrected masks, the backend service packages them into a new, versioned data asset. These assets are automatically named with a convention that includes the user's ID and a timestamp (e.g., `axonrouter_student_corrections_Endijs_20250626_040517:1`), providing clear lineage for when and from whom new training data was sourced. These data assets serve as direct inputs to the Azure ML training and evaluation pipelines. By referencing a data asset (e.g., `azureml:asset_name:version`), the pipelines can reliably fetch the exact version of the data they need to run, which is critical for comparing model performance and debugging training runs over time.

Access & Permissions

Access to the Azure Machine Learning workspace and its associated resources (like Azure Blob Storage for datastores) is managed securely through a non-interactive identity, ensuring that no personal user credentials are required for automated processes.

Service Principal Authentication: All automated workflows, including the CI/CD/CT pipelines in GitHub Actions and the running Docker containers (FastAPI backend and Streamlit frontend), authenticate with Azure using a Service Principal. This is an application identity within Microsoft Entra ID (formerly Azure Active Directory) that is granted specific permissions to perform tasks on behalf of the application.

Credential Management: The credentials for the service principal (Tenant ID, Client ID, and Client Secret) are managed as secure environment variables.

Local Development: For local container testing, credentials are loaded from a .env file, which is excluded from version control via .gitignore.

Production (CI/CD): In the GitHub Actions workflow, these credentials are stored as encrypted repository secrets and are safely injected into the pipeline jobs at runtime.